MINI-PROJECT — PART N°1

Linux System Audit and Monitoring Using Shell Scripting

"Design and Implementation of an Automated Hardware & Software Audit System with Reporting and Remote Monitoring Capabilities"

Student Name	Belouadah Saad Eddine Koussai
Section	Section B — Group 3
Institution	National School of Cyber Security (NSCS)
Teacher	Dr. BENTRAD Sassi
Submission Date	March 30, 2026
Academic Year	2025 / 2026

Language: Bash Shell	OS: Linux (Ubuntu / Kali)	Scripts: 9 Modules

1. Project Overview

This project was developed as part of the NSCS Operating Systems course for Academic Year 2025/2026. The objective is to design and implement a fully automated Linux-based system audit and monitoring solution using Bash shell scripting. The system collects detailed hardware and software information, generates structured reports in multiple formats, transmits reports via email, automates execution through cron jobs, and supports remote monitoring via SSH.

The solution is designed with modularity and security in mind — each functional area is encapsulated in a dedicated script module, following Cybersecurity best practices for system inventory and monitoring.

2. System Architecture & Design

The system follows a modular architecture, separating concerns into distinct script files. The main entry point **audit_main.sh** orchestrates all modules via function calls. Below is the complete module structure:

Script File	Role / Responsibility	Key Functions
audit_main.sh	Main entry point & orchestration	main(), run_audit(), interactive_menu()
config.sh	Global configuration variables	Paths, email, SSH, thresholds
lib_colors.sh	Terminal colors & logging utilities	color_echo(), log_info/warn/error()
hw_audit.sh	Hardware information collection	collect_hw_info() → HW[] array
sw_audit.sh	OS & software information collection	collect_sw_info() → SW[] array
report.sh	Report generation (TXT/HTML/JSON)	generate_report()
email.sh	Email transmission module	send_report_email(), _build_mime()
remote.sh	Remote monitoring via SSH/SCP	push_report_remote(), pull_remote_audit()
setup_cron.sh	Cron automation & log rotation	install_cron(), rotate_logs(), verify_report()

Data Flow

When **audit_main.sh** is executed, it sources all modules, then calls **collect_hw_info()** and **collect_sw_info()** which populate associative arrays HW[] and SW[]. The **generate_report()** function then reads these arrays and writes formatted output in TXT, HTML, and JSON formats simultaneously. Finally, optional email dispatch and remote push are triggered.

The startup_prompts() function provides an interactive wizard on first run, allowing the user to select report type, email recipient, and remote target — even without passing command-line flags.Reality: it runs every time you execute ./audit_main.sh --full without passing --email and --remote flags. It's not "first run" — it's any run without those specific flags.

3. Key Commands & Tools Used

3.1 Hardware Audit Commands

Command	Purpose
/proc/cpuinfo	CPU model, cores, frequency, cache
nproc --all	Total logical CPU count
uname -m / uname -r / uname -a	Architecture, kernel version, full kernel info
/proc/meminfo, free -h	RAM total, free, available; swap details
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT	Full disk layout with filesystems
df -hT	Disk usage per partition with type
ip addr / ip link / ip route	Network interfaces, MAC addresses, default gateway
/etc/resolv.conf	DNS server configuration
dmidecode	Motherboard vendor/product, BIOS info, serial number
lsusb	Connected USB devices
lspci	PCI devices including GPU detection
top -bn1	CPU usage snapshot
uptime -p	System uptime in human-readable format

3.2 Software & OS Audit Commands

Command	Purpose
/etc/os-release	OS name, version, ID
dpkg -l / rpm -qa / pacman -Q	Installed packages (multi-distro support)
who / last -n 10 / lastb -n 10	Logged-in users, recent logins, failed logins
systemctl list-units --state=active/faile	Rdunning and failed services
ps aux --sort=-%cpu | head -21	Top 20 processes by CPU usage
ss -tuln / netstat -tuln	Open TCP/UDP listening ports
ufw / firewall-cmd / iptables -L	Firewall status (auto-detected)
crontab -l / ls /etc/cron.d/	Scheduled tasks
find / -perm -4000 -type f	SUID binaries security check
find /etc /tmp /var -perm -0002 -type d	World-writable directories check
timedatectl	Timezone information
locale	System locale / language

4. Cron Automation Configuration

The system automates daily audit execution using the Unix **cron** daemon. The **setup_cron.sh** script handles installation, removal, and log rotation. The schedule is configured in **config.sh** and defaults to **daily at 04:00 AM**.

Cron Schedule Configuration (config.sh)

```
CRON_SCHEDULE="0 4 * * *" # Run daily at 04:00 AM
```

Installing the Cron Job

Run the setup helper once to install the cron entry:

```
sudo bash setup_cron.sh install
```

Generated Cron Entry

```
0 4 * * * /path/to/audit_main.sh --full >> /var/log/sys_audit/logs/cron_exec.log 2>&1 #
sys_audit — managed entry
```

Verify Installation

```
crontab -l
```

Cron Expression Breakdown

Field	Value	Meaning
Minute	0	At minute 0
Hour	4	At 04:00 AM
Day of Month	*	Every day
Month	*	Every month
Day of Week	*	Any day of week

Log Rotation Reports older than **30 days** are automatically pruned. The setup_cron.sh rotate command uses **find** with -mtime +30 -delete to clean both the reports and logs directories. An optional **logrotate** configuration is also written to /etc/logrotate.d/sys_audit for daily compression and 30-day retention.

5. Security Considerations

- **Report Integrity Verification**

  Eachg eneratedreportisaccompanied by a **SHA-256 hash file** (report.sha256). The setup_cron.sh verify <report> command re-computes the hash and compares it to detect tampering.

- **SSH Key-Based Authentication**
  Remotep ushoperations(remote.sh)useSSH with **BatchMode=yes** and **StrictHostKeyChecking=no** for automation. In production, SSH_KEY should point to a dedicated key pair with limited permissions (read-only remote user).
- **SUID Binary Detection**
  The software audits cansfor SUID-set files using find / -perm -4000 -type f. Unexpected SUID binaries can indicate privilege escalation vectors.
- **World-Writable Directory Audit**
  Criticald irectories(/etc,/tmp,/var)are scanned for world-writable permissions, which represent a common misconfiguration risk.
- **Firewall Status Collection**
  Thes ystemauto-detectsandrecords firewall state (ufw, firewalld, or iptables), providing a snapshot of the network security posture.
- **Failed Login Monitoring**
  Thel astbcommandcapturesthe 10 most recent failed login attempts, useful for detecting brute-force activity.

- **CPU Alert System**
  IfC PUusage exceedsthe configurable threshold (default: **80%**), an alert email is dispatched automatically via the check_cpu_alert() function.
- **Sensitive Data in Reports**
  Reports mayc ontainsensitive system information. The REPORT_DIR should be protected with chmod 700 and owned by root. Email transmission uses MIME with authenticated SMTP (msmtp) to prevent cleartext exposure.

6. Challenges Encountered

#1 Multi-Distribution Compatibility
Different Linux distributionsusedifferentpackage managers (apt/dpkg, rpm/dnf, pacman). The solution uses conditional command detection with command -v to gracefully fall back and report the appropriate package manager.

#2 Root Privileges for DMI/Hardware Info
dmidecode requires root toaccessBIOS/motherboard data. The script detects the effective UID with $EUID and falls back to a descriptive message when running as a non-root user, avoiding permission errors.

#3 CPU Usage Parsing
The top-bn1outputformat varies slightly across distributions. The awk expression '{print 100 - $8 "%"}' correctly targets the idle percentage column, but the column number can differ. A robust fallback was added.

#4 **Email with HTML/Text Multipart MIME**
Constructingavalid MIME emailwithbothanHTML body and a plain-text attachment required careful boundary handling in the _build_mime() function to ensure compatibility with various mail clients.

#5 **Associative Arrays in Bash**
Usingdeclare-Aassociativearraysrequired Bash version 4+. Scripts include #!/usr/bin/env bash shebang to ensure the correct interpreter is used, avoiding compatibility issues with older /bin/sh.

#6 **SSH Automation Without Passwords**
Configuringpasswordless SSHfor remotepushrequired setting up key-based authentication and using BatchMode=yes to prevent interactive prompts from breaking automated cron execution.

7. AI uses

AI was used :

- Defining the overall project structure
- resolving some technical issues and errors in the code that were not apparent
- adding comments to the project.
- help me in command cocumentation

8. Bonus Features Implemented

Bonus Feature	Implementation	Location
■ Colorized Terminal Output	ANSI color codes via color_echo() for all status messages	lib_colors.sh
■ Interactive Menu (select)	8-option menu driven by bash select; startup wizard with 3-step flow	audit_main.sh
■ Log Rotation Mechanism	find -mtime +30 -delete + //etc/logrotate.d/ config generation	setup_cron.sh
■ SHA-256 Integrity Verification	sha256sum hash per report; handled by verify_report()	setup_cron.sh
■ CPU Alert System	Threshold-based alert (>80% CPU); handled by check_cpu_alert()	setup_cron.sh
■ Report Diff / Change Detection	diff -color=always; handled by diff_reports()	setup_cron.sh
■ Remote Monitoring Loop	SSH polling loop; handled by monitor_remote_loop()	remote.sh
■ Multi-Format Reports	TXT, HTML, JSON outputs from single run	report.sh
■ Multi-Format Reports		report.sh
■ Multi-Format Reports	TXT, HTML, JSON outputs from single run	report.sh

9. Email Configuration Documentation

The email module supports three tools: **msmtp** (default), **mail/mailx**, and **sendmail**. The active tool is set via the MAIL_TOOL variable in **config.sh**.

msmtp Configuration (~/.msmtprc)

```
accountdefault  host smtp.gmail.com port 587 auth on tls on tls_starttls on user
your_email@gmail.com password your_app_password from your_email@gmail.com logfile
~/.msmtp.log
```

The email is sent as a **multipart/mixed MIME** message with an HTML body (rendered in the email client) and the plain-text report as an attachment. The _build_mime() function in **email.sh** constructs
the full MIME envelope before piping it to the mail tool. The recipient address is validated with a basic regex before dispatch.

10. Remote Monitoring Implementation

The **remote.sh** module implements three remote monitoring capabilities:

push_report_remote() ***SCP-based report push***	Transfers TXT, HTML, JSON, and SHA-256 hash files to a centralized remote directory via SCP. Creates the remote directory via SSH if it does not exist.
run_remote_audit_single() / run_remote_audit_all() Live remote execution & aggregation:	Iterates over configured remote hosts, securely uploads the project's audit modules (hw_audit.sh & sw_audit.sh) to the target via SCP, executes them over SSH to generate a comprehensive report, saves the output locally on the Admin PC, and finally cleans up the temporary files on the remote server.

11. Conclusion

This project successfully implements a complete, modular Linux system audit and monitoring solution meeting all required functional specifications. The system collects comprehensive hardware and software information, generates professionally formatted reports in multiple output formats, transmits reports via email using MIME-compliant multipart messages, automates execution through cron scheduling, and supports remote monitoring via SSH.

Beyond the base requirements, all documented bonus features have been implemented: colorized output, interactive menu, log rotation, SHA-256 integrity verification, CPU alert system, report change detection, and continuous remote monitoring loop.

This project reinforces core Cybersecurity skills including system inventory management, secure remote access, automated threat detection, and audit trail maintenance — all foundational competencies for a Security Operations Center (SOC) analyst or system administrator.